DATA MINING SEMINARS TOPICS

Recommended 15-Minute Presentation Timeline

Time	Phase	What to Do
Min 1–3	**Introduction**	Introduce the tool/platform & the real-world problem it solves.
Min 4–8	**Mining Technique**	Explain the data mining algorithm: how it works, inputs, outputs.
Min 9–12	**Code Demo**	Run and explain the Python/code snippet live. Show output.
Min 13–15	**Conclusion & Q&A**	Summarise findings, limitations, and invite questions.

General Advice for All Presentations

☑ Do This

Start with a real headline statistic (e.g., 'Netflix saves $1B/year with this algorithm').
Run your code live — even a small output proves you understand it.
Visualise your data: a chart is worth 10 slides of bullets.
Connect the algorithm to the business outcome — examiners want both.
Rehearse the 15-minute timing at least twice before the seminar day.

⚠ Avoid This

Reading from slides — know your project; speak, don't recite.
Skipping the dataset — always name the dataset and show it loading.
Over-explaining code line-by-line — explain the 'what' and 'why', not every syntax detail.
Ignoring limitations — every algorithm has weaknesses; discuss them confidently.
Presenting without a conclusion — always end with: 'What did we learn? What would we do next?'

01 Waze — Real-Time Traffic Pattern Mining

Description	Waze crowdsources GPS pings from millions of drivers to detect traffic anomalies (accidents) and congestion clusters in real time, then re-routes drivers using graph shortest-path algorithms.
Dataset	Simulated GPS log: list of (latitude, longitude, speed_kmh, timestamp) rows — or the public Uber Movement dataset (movement.uber.com). ~500 rows minimum.
ML Algorithm	K-Means Clustering (detect congestion zones) + Anomaly/Outlier Detection (Isolation Forest for accident spikes) + Dijkstra's Algorithm (re-routing).
Tools & Libs	Python · Pandas · Scikit-learn (KMeans, IsolationForest) · Folium (map visualisation) · NetworkX (Dijkstra graph).

What the Student Should Present

1. Show a map of Lagos/your city with GPS pings plotted.
2. Run K-Means — highlight the 3 congestion cluster centroids on the map.
3. Run Isolation Forest — flag the anomalous speed dip (accident zone).

02 Siri / Alexa — Voice Pattern Mining & Intent Classification

Description	Virtual assistants convert raw audio waveforms into numerical feature vectors (MFCCs) and then classify the spoken intent (e.g., 'Set alarm' vs 'Play music') using a supervised classifier.
Dataset	Google Speech Commands dataset (Kaggle) — 65,000 one-second WAV clips across 30 command labels. Alternatively, record 10 personal audio clips per command class.
ML Algorithm	Feature Extraction: Mel-Frequency Cepstral Coefficients (MFCCs). Classification: Random Forest or Support Vector Machine (SVM) on MFCC feature vectors.
Tools & Libs	Python · Librosa (audio feature extraction) · Scikit-learn (RandomForestClassifier / SVM) · Matplotlib (waveform + spectrogram plots).

What the Student Should Present

1. Play a sample audio clip; show the raw waveform plot.
2. Explain what MFCCs are — 'the fingerprint of sound.'
3. Show the MFCC heatmap (time vs frequency vs amplitude).
4. Train and test a Random Forest on pre-computed MFCC features.
5. Show classification report: precision/recall per intent class.
6. Discuss: why accents and background noise lower accuracy.

03 Sentiment Mining in Social Media for Brand Health

Description	Mining thousands of product-related posts/tweets to measure whether public sentiment is positive, negative, or neutral — giving businesses a real-time 'brand health score.'
Dataset	Twitter US Airline Sentiment dataset (Kaggle, 14,000 tweets) or Amazon Product Reviews dataset. Both contain text + pre-labelled sentiment for validation.
ML Algorithm	Rule-based baseline: TextBlob/VADER Sentiment Analysis. Advanced: Fine-tuned BERT for Sentiment Classification (binary or 3-class).
Tools & Libs	Python · TextBlob · VADER (NLTK) · Pandas · Matplotlib / Seaborn (sentiment distribution charts) · WordCloud.

What the Student Should Present

1. Load 50 sample tweets about a brand (e.g., MTN, Airtel, iPhone).
2. Run TextBlob polarity scoring — print positive / negative / neutral counts.
3. Show a bar chart: sentiment distribution across the dataset.
4. Generate a WordCloud of the most frequent positive and negative words.
5. Show how a brand could act: 'We see 62% negative sentiment → product recall risk.'
6. Discuss: sarcasm as the biggest NLP challenge.

04 Social Media Trend Detection (Twitter/X)

Description	Platforms like Twitter detect trending topics by mining hashtag co-occurrence, tweet velocity (rate of new posts per minute), and geographic clustering of similar terms.
Dataset	Trending Twitter dataset (Kaggle) or a custom CSV of 200 sample tweets with hashtags. A sample file is easy to create manually for demo purposes.
ML Algorithm	Hashtag Frequency Analysis (Counter/TF) · TF-IDF for term importance
Tools & Libs	Python · Pandas · Collections (Counter) · Scikit-learn (TfidfVectorizer, LDA) · Matplotlib / Seaborn (bar charts) · WordCloud.

What the Student Should Present

1. Load the tweet dataset and show the raw hashtag list.
2. Run Counter — display Top 10 trending hashtags as a bar chart.
3. Apply TF-IDF — explain why TF-IDF ranks hashtags better than raw count.
4. Map the top trend to a real-world event (e.g., #WorldCup spike).
5. Discuss: how bots artificially inflate trending counts (data quality problem).

05 n8n — Building Automated Data Mining Pipelines

Description	n8n is a no-code workflow automation tool that implements ETL (Extract, Transform, Load) pipelines — automatically mining data from Gmail, APIs, or databases and routing it to dashboards or CRMs without manual work.
Dataset	Live API feed: OpenWeatherMap free API (weather data every 10 min) or a public RSS news feed. Alternatively, simulate a pipeline with a CSV of 100 email subjects + labels.
ML Algorithm	ETL Pipeline Logic · Rule-Based Routing (if/else nodes) · Optional: Naive Bayes text classification on email subjects (spam vs. important) integrated as a Python function node.
Tools & Libs	n8n (no-code UI — show screenshots) · Python · Requests (API calls) · Pandas · JSON · Optional: NLTK Naive Bayes for the classification step.

What the Student Should Present

1. Diagram the ETL flow: Source (Gmail/API) → Extract → Transform → Load (Google Sheet / DB).
2. Show an n8n workflow screenshot with at least 3 nodes connected.
3. Explain each node type: Trigger, HTTP Request, Set/Transform, Database Write.
4. Run the Python simulation showing data flowing through extract → classify → store.
5. Show the output CSV/sheet with classified email labels.
6. Discuss: how enterprises chain 50+ nodes to automate entire business workflows.

06 Automated Lead Scoring for Businesses

Description	Businesses mine 'behavioural intent signals' (page visits, email opens, download counts) to score prospects and predict purchase likelihood — allowing sales teams to focus on the hottest leads.
Dataset	HubSpot CRM Sample Data (Kaggle) or a synthetic dataset of 500 leads with features: pages_visited, emails_opened, time_on_site, downloads, converted (0/1).
ML Algorithm	Logistic Regression (primary — interpretable probability output) + Feature Importance analysis to show which signals matter most.
Tools & Libs	Python · Pandas · Scikit-learn (LogisticRegression, train_test_split, classification_report) · Matplotlib (feature importance bar chart).

What the Student Should Present

1. Show the lead dataset: each row is a prospect, each column is a behaviour signal.
2. Explain Logistic Regression — outputs a probability 0–1 ('How likely to buy?').
3. Train the model; show accuracy score and classification report.
4. Plot feature importances — which signal (e.g., 'pages_visited') predicts conversion best?
5. Demo: input a new lead's data and get a live score (0.87 = 87% likely to convert).

07 Netflix — Collaborative Filtering & Recommendation

Description	Netflix predicts what you will watch next by finding users with similar taste profiles, then recommending items those users loved. This is User-Based Collaborative Filtering.
Dataset	MovieLens Small dataset (100,000 ratings, 9,000 movies, 600 users) — available free at grouplens.org. Or use the Netflix Prize sample on Kaggle.
ML Algorithm	User-Based Collaborative Filtering (Cosine Similarity on User-Item Matrix) + Matrix Factorization via SVD (Singular Value Decomposition) using Surprise library.
Tools & Libs	Python · Pandas (pivot_table) · Scikit-learn (cosine_similarity) · Surprise library (SVD) · Seaborn (heatmap of the rating matrix).

What the Student Should Present

1. Build and display the User × Movie rating matrix (pivot table) as a heatmap.
2. Explain the 'cold start problem' — a new user has no data to base recommendations on.
3. Compute cosine similarity between users — show the most similar user pairs.
4. Recommend top 3 unseen movies to a target user based on their nearest neighbour.

08 Spotify — Content-Based Filtering for Discover Weekly

Description	Spotify mines the raw acoustic features of every song (tempo, energy, valence, danceability) and uses K-Nearest Neighbours to find songs that are numerically similar — regardless of genre labels.
Dataset	Spotify Tracks Dataset (Kaggle, 600,000 tracks with 18 audio features) or the smaller 'Spotify Song Attributes' dataset (~2,000 songs).
ML Algorithm	K-Nearest Neighbours (KNN) for similarity search on the audio feature space. Feature Engineering (StandardScaler normalisation) is essential before running KNN.
Tools & Libs	Python · Pandas · Scikit-learn (NearestNeighbors, StandardScaler) · Matplotlib / Seaborn (radar chart of audio features).

What the Student Should Present

1. Show the audio feature table: each row = 1 song, columns = tempo, energy, valence…
2. Explain why normalisation is critical: tempo (120 BPM) would dominate energy (0.8) without it.
3. Run KNN — input a 'seed song' and display the 5 nearest songs by feature distance.
4. Plot a radar/spider chart comparing the seed song vs its recommendations.
5. Contrast with Collaborative Filtering: 'Content-Based doesn't need user history — it works for new users.'
6. Discuss: Spotify's real system uses a hybrid of both approaches.

9 Healthcare — Mining Patient Data for Disease Prediction

Description	Hospitals mine Electronic Health Records (EHR), lab values, and wearable sensor readings to train predictive models that identify high-risk patients before symptoms worsen — potentially saving lives.
Dataset	Pima Indians Diabetes Dataset (Kaggle, 768 patients, 8 clinical features, binary outcome). Also: Breast Cancer Wisconsin Dataset (Scikit-learn built-in).
ML Algorithm	Random Forest Classifier (primary — handles mixed features, gives feature importance). Also compare: Logistic Regression as baseline. Evaluate with Confusion Matrix + ROC-AUC.
Tools & Libs	Python · Pandas · Scikit-learn (RandomForestClassifier, ConfusionMatrixDisplay, roc_auc_score) · Matplotlib · Seaborn.

What the Student Should Present

1. Explore the dataset: show class imbalance (healthy vs diabetic counts).
2. Train Random Forest; display the confusion matrix and explain TP/FP/FN/TN.
3. Plot feature importances — which clinical value (e.g., 'Glucose') is most predictive?
4. Discuss: False Negatives are more costly in healthcare than False Positives.
5. Show ROC curve and explain AUC score.
6. Discuss ethics: bias in training data if dataset over-represents one demographic.

10 HR Tech — Mining Employee Data for Talent Analytics

Description	HR departments mine employee data (satisfaction scores, promotion history, overtime patterns, tenure) to predict which employees are at risk of leaving — enabling targeted retention interventions before resignations happen.
Dataset	IBM HR Employee Attrition Dataset (Kaggle, 1,470 employees, 35 features, binary label: Attrition Yes/No). One of the most widely used HR analytics benchmarks.
ML Algorithm	Gradient Boosting Classifier (XGBoost or scikit-learn GradientBoostingClassifier) for high accuracy. SHAP values or feature importance for interpretability.
Tools & Libs	Python · Pandas · Scikit-learn (GradientBoostingClassifier, LabelEncoder) · Matplotlib · Seaborn (correlation heatmap, attrition rate by department).

What the Student Should Present

1. Show the IBM dataset; highlight the class imbalance (only ~16% attrition).
2. Encode categorical features (Department, Gender) using LabelEncoder.
3. Train Gradient Boosting; show accuracy and classification report.
4. Plot top 10 features driving attrition — is it overtime? Low salary? Long commute?
5. Show attrition rate by department as a bar chart.
6. Discuss: fairness concerns — should age or marital status be used as a predictor?

11 Supply Chain — Mining Logistics Data for Demand Forecasting

Description	Retailers mine years of point-of-sale data, weather records, and calendar events to forecast future demand — preventing both costly overstock and revenue-damaging stockouts across thousands of product SKUs.
Dataset	Rossmann Store Sales dataset (Kaggle, 1M+ rows of daily sales for 1,115 stores) or Store Item Demand Forecasting Challenge (Kaggle). A 3-year monthly subset works for demos.
ML Algorithm	ARIMA / SARIMA (classical time-series, shows seasonality). Facebook Prophet (handles holidays automatically). XGBoost Regression (feature-based, most accurate in practice).
Tools & Libs	Python · Pandas · Statsmodels (ExponentialSmoothing / ARIMA) · Prophet (pip install prophet) · Matplotlib (actual vs forecast line chart).

What the Student Should Present

1. Plot 24 months of sales data — visually identify the seasonal pattern.
2. Decompose the series: trend + seasonality + residual components.
3. Fit Holt-Winters Exponential Smoothing; forecast 3 months ahead.
4. Plot actual vs forecasted values with a confidence interval band.
5. Explain the 'Bullwhip Effect' — how small demand fluctuations amplify up the supply chain.
6. Discuss: how Walmart shares real-time sales data with suppliers to dampen the Bullwhip Effect.